

Control Structures: Loops and Conditionals

In this lecture, we will explore control structures in Python, specifically focusing on loops and conditional statements. Control structures are essential for implementing logic in programming, allowing us to control the flow of execution based on certain conditions. By the end of this lecture, you will understand how to use loops to repeat actions and conditionals to execute code based on specific criteria.

Introduction to Control Structures

Control structures in programming allow us to dictate how our code executes under various circumstances. In Python, the two primary control structures we will discuss today are loops and conditionals.

Conditionals

Conditionals allow us to execute certain pieces of code based on whether a condition is true or false. The most common conditional statements in Python are `if`, `elif`, and `else`.

The `if` Statement

The `if` statement executes a block of code if a specified condition is true. The syntax for the `if` statement is as follows:

```
if condition:
    statements
```

Example:

```
gpa = 3.4
if gpa > 2.0:
    print("Your application is accepted.")
```

In this example, the message will be printed only if the GPA is greater than 2.0.

The `if/else` Statement

The `if/else` statement allows us to execute one block of code if the condition is true and another block if it is false. The syntax is:

```
if condition:
    statements
else:
    statements
```

Example:

```
gpa = 1.4
if gpa > 2.0:
    print("Welcome to Mars University!")
else:
    print("Your application is denied.")
```

This will print a different message based on the value of `gpa`.

The `elif` Statement

The `elif` statement allows us to check multiple conditions in sequence. The syntax is:

```
if condition1:
    statements
elif condition2:
    statements
```

```
else:  
statements
```

Example:

```
x = 30  
if x <= 15:  
y = x + 15  
elif x <= 30:  
y = x + 30  
else:  
y = x  
print('y =', y)
```

Loops

Loops are used to execute a block of code repeatedly, either a specific number of times or until a certain condition is met. In Python, we primarily use for loops and while loops.

The for Loop

The for loop is used to iterate over a sequence (like a list or a range). The syntax is:

```
for variable in iterable:  
statements
```

Example:

```
for x in range(1, 6):  
print(x, "squared is", x * x)
```

This will output the squares of the numbers from 1 to 5.

The range() Function

The range() function generates a sequence of numbers. It can take one, two, or three arguments: - range(stop) - generates numbers from 0 to stop-1. - range(start, stop) - generates numbers from start to stop-1. - range(start, stop, step) - generates numbers from start to stop-1, incrementing by step.

Example:

```
for x in range(5, 0, -1):  
print(x)  
print("Blastoff!")
```

This will count down from 5 to 1.

The while Loop

The while loop repeatedly executes a block of code as long as a specified condition is true. The syntax is:

```
while condition:  
statements
```

Example:

```
number = 1  
while number < 200:  
print(number)  
number = number * 2
```

This will print powers of 2 until the number reaches or exceeds 200.

Logic and Operators

When working with conditionals and loops, we often use logical and relational operators: - **Relational Operators:** ==, !=, <, >, <=, >= - **Logical Operators:** and, or, not

Example:

```
if x < 10 and x > 0:  
    print("x is a positive single-digit number.")
```

Loop Control Statements

Sometimes, we need to control the flow of loops more precisely. Python provides control statements like break, continue, and pass: - **break:** Exits the closest enclosing loop. - **continue:** Skips the current iteration and jumps to the next iteration of the loop. - **pass:** Does nothing; it is a placeholder statement.

Example of break and continue:

```
for i in range(10):  
    if i == 5:  
        break # exits the loop when i is 5  
    if i % 2 == 0:  
        continue # skips even numbers  
    print(i)
```

Conclusion

In this lecture, we covered the fundamental control structures in Python: loops and conditionals. You learned how to use if, elif, and else statements to implement decision-making in your programs, as well as how to utilize for and while loops to repeat actions. Understanding these control structures is crucial for writing effective and efficient Python code. In the next lecture, we will dive deeper into functions and how they can help organize and modularize our code.

Quiz Questions

Q1: What are the primary control structures discussed in the lecture?

- Functions and Classes
- Loops and Conditionals
- Variables and Data Types
- Modules and Packages

Correct: Loops and Conditionals

Q2: Which statement is used to execute a block of code if a condition is false?

- if
- elif
- else
- for

Correct: else

Q3: What does the `break` statement do in a loop?

- Skips the current iteration
- Exits the closest enclosing loop
- Pauses the loop execution
- Continues to the next iteration

Correct: Exits the closest enclosing loop